Anti-Pattern: Hidden State in Cells — A Standalone Formalization of the Cell-Internal-State Source-of-Truth Failure in the Coordination Knowledge Substrate Pattern

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 6, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone publication, the architectural anti-pattern called *hidden state in cells* — the deployment configuration in which cell-internal state operationally determines coordination outcomes — so that downstream readers can identify the failure mode, distinguish it from related anti-patterns, and apply the architectural correction the source paper commits to.

Abstract

The CKS pattern commits to two foundational properties that, together, locate authority over coordination outside any cell: the substrate-cell boundary, which separates substrate (state) from cells (behavior); and the substrate as source of truth, which makes substrate the authoritative location for coordination-relevant state. *Hidden state in cells* is the deployment configuration in which both commitments fail through a single operational mechanism: cells maintain internal state — local variables persisting across invocations, runtime caches, session state, internal buffers, service-level state, instance data, runtime configuration — that operationally determines coordination outcomes invisible to substrate inspection. This note formalizes the anti-pattern as four operational components, names the CKS commitments it violates, traces the failure mode, specifies the architectural correction, distinguishes it from four adjacent legitimate patterns, and provides an operational test with three sharpening properties. The anti-pattern is the *general* cell-internal-state source-of-truth failure: it generalizes the agent-memory-specific case and the LLM-context-specific case to all cell-internal state that affects coordination across executions, and it compounds with the cell-as-substrate architectural-boundary failure when cell-held state is also operationally authoritative.

1. Why a standalone formalization is needed

The CKS pattern's foundational commitments to the substrate-cell boundary (§2.1) and to the substrate as source of truth (§11.3) together ensure that coordination decisions are determined by substrate state, not by state held inside the components that execute against substrate. The two commitments operate on the same underlying architectural move: state lives in substrate; cells execute behavior over

substrate state without holding state of their own. When a deployment violates this move — when cells acquire and retain state that affects coordination — it does not merely fail one commitment. It fails both, simultaneously, through the same operational mechanism.

The motivating cases are deployments in which cell-internal state shapes coordination behavior: a cell that maintains a counter across invocations and routes work based on its current value; a cell that caches computed results internally and serves them to downstream operations without substrate read; a session-based cell whose behavior depends on accumulated session state; a service-based cell whose instance data has drifted from its initial configuration; a cell whose runtime configuration variable was set at startup and is consulted by every invocation thereafter. Each instance holds coordination-relevant state inside the cell. None of that state is visible in substrate, and none of it is governable by humans inspecting substrate.

A standalone treatment is necessary on three grounds. The anti-pattern is the *general* failure mode for cell-internal-state source-of-truth migration: the agent-memory-specific case and the LLM-context-specific case are formalized separately, but neither covers a non-LLM cell whose instance variables hold a coordination-relevant counter. The anti-pattern is operationally common: stateful service architectures, language frameworks with closure semantics, and session-based AI systems naturally accumulate cell-internal state, and when such patterns are applied to coordination-affecting state without preserving substrate as the source of truth, the deployment exhibits hidden state in cells by default. And the strategic posture of public prior art: derivations that propose stateful AI cells, closure-based AI processing, or session-stateful AI architectures are substantially more contestable when the anti-pattern is publicly formalized as standalone.

2. The anti-pattern, defined precisely

A deployment exhibits *hidden state in cells* when all four of the following operational components hold.

(a) Cell-internal state persists across executions and affects coordination. The cell maintains internal state — local variables, runtime caches, session state, instance data, configuration variables, internal buffers — that persists between invocations. The state is cell-internal, not substrate-recorded. Its persistence affects how subsequent invocations of the cell behave for coordination purposes.

(b) Cell behavior depends on cell-internal state not visible in substrate. Cell outputs depend on the cell's internal state at invocation time. Identical substrate inputs may produce different cell outputs depending on the cell's accumulated internal state. The deployment's coordination behavior is shaped by state invisible to substrate inspection.

(c) Downstream operations consult cell behavior shaped by hidden state. Operations that depend on coordination outcomes consume cell outputs. Those outputs reflect hidden cell state. The architectural commitment that downstream operations consult substrate as the source of truth fails: they are operationally consulting cell-state-shaped outputs whose authoritativeness derives from state outside substrate.

(d) Cell-state changes occur without substrate-recording. Internal state transitions within cells — counter increments, cache updates, session-state advances, instance-data modifications, configuration

adjustments — occur without producing substrate writes. The architectural commitment that coordination-relevant state changes are substrate-recorded fails for cell-internal state changes.

A deployment exhibiting any one of (a)–(d) partially exhibits the anti-pattern; a deployment exhibiting all four exhibits it fully.

3. Which CKS commitments are violated

Hidden state in cells violates two foundational commitments simultaneously and produces cascade violations through several decompositions.

The substrate-cell boundary commitment is directly violated because cells hold state. The substrate-as-source-of-truth commitment is directly violated because cell-internal state operationally determines coordination outcomes that should be substrate-determined. Both fail through the same operational mechanism.

Within the substrate-cell boundary decomposition, the commitment that *substrate holds state* is directly violated when cells hold coordination-relevant state, and the commitment that *cells execute behavior* is operationally compromised because cells are no longer pure behavior over substrate input but behavior over substrate input plus internal state.

Within the source-of-truth decomposition, the five categories of authoritative substrate state are each potentially violated when hidden cell state determines authoritative answers within that category. The category covering "what is the case" fails when hidden cell state determines what is the case for downstream operations. The category covering "what is current" fails when hidden cell state determines what is current. Categories covering "what was decided," "what is permitted," and "what was attempted" are violated whenever cell-internal state holds the authoritative answers for them.

For LLM-based cells, the AI-as-substrate-mediator commitment that the LLM does not hold substrate-relevant state outside substrate is directly violated; the commitment extends to all cell-internal state for cells whose execution is LLM-mediated.

The cascade implications across other foundational commitments are: the determinism contract is extended-violated because cell behavior depends on cell-internal state in addition to substrate state, which compromises read determinism (the guarantee that reads of authoritative content are consistent given substrate state); path retraceability is extended-implicated because decisions made by cells based on hidden state cannot be retraced through the substrate trail; the AI-as-substrate-mediator commitment is extended-implicated for LLM-based cells through Property C; and the human-governed commitment is extended-implicated because humans inspecting substrate cannot inspect hidden cell state, so the inspect right is operationally compromised at the cell-internal-state layer. Per-substrate human governance preservation, the composition requirement that any multi-substrate composition preserve governance, is extended-violated when governance does not extend to cell-internal state but cell-internal state operationally exercises authority.

4. The failure mode

Hidden state in cells produces deployments in which cell behavior depends on state invisible to substrate. The downstream consequences are operationally specific.

Cell behavior varies with accumulated state. Over a deployment's lifecycle, cells accumulate internal state — counters increment, caches grow, session state advances, instance data drifts. The same substrate input may produce different cell outputs depending on when the cell is invoked relative to its accumulated state. Reproducibility is operationally compromised; identical substrate state produces different cell outputs across instances or across time. Read determinism fails because cell outputs depend on cell-internal state in addition to substrate state, and change addressability is operationally compromised because changes can occur in cell-internal state without corresponding substrate change.

Decisions cannot be retraced through substrate. The retraceable trail records substrate state changes; decisions made by cells based on hidden state cannot be reconstructed because the determining state is not substrate-recorded, so the trail has gaps at hidden-state-driven moments. Humans exercising the inspect right inspect substrate; they cannot inspect cell-internal state, so the deployment operates with coordination behavior shaped by state invisible to governance.

Hidden-state-driven emergent behavior arises. Cells with accumulated state may exhibit behavior that no orchestration rule specifies. The deployment may operate effectively in nominal cases but exhibit unexpected behavior driven by hidden-state interactions that no rule predicts and no audit can identify. Distributed deployments with multiple cell instances may have instances that accumulate different state, so coordination behavior depends on which instance is consulted; the architectural commitment to substrate-determined coordination fails through inter-instance divergence.

Recovery from hidden-state errors is operationally constrained: identifying the cause requires inspecting cell-internal state, which is operationally difficult or impossible for cells with complex internal state, and recovery may require restarting cells, which loses accumulated state and causes operational disruption. Over a deployment's lifecycle, cell-internal state may drift from any intended configuration, and the deployment operates with cells whose state has accumulated unexpectedly; the determinism commitment fails through drift even if no individual invocation is incorrect.

The anti-pattern compounds with the cell-as-substrate anti-pattern at the architectural-boundary layer: cell-as-substrate is the failure of the substrate-cell boundary in which cells hold state outside substrate; hidden state in cells is the source-of-truth failure when that held state operationally determines coordination. The two compound when cells hold coordination-relevant state that is also operationally authoritative. The anti-pattern also compounds with the agent-memory and LLM-context specific cases when those cell-internal-state forms are present.

5. The architectural correction

The architectural correction operates through three foundational commitments together: the substrate-cell boundary (cells do not hold state across executions), substrate as source of truth (coordination-affecting state lives in substrate), and the determinism contract (cell behavior is deterministic from substrate input alone).

Cells must be stateless or have only ephemeral within-execution state. Cell behavior must depend only on substrate input and on within-execution working state that is cleared at execution end. Stateful cell

architectures must be re-architected as stateless: where a stateful service architecture is in use, its coordination-relevant state must migrate from cell instances to substrate; the cells become readers and writers of substrate rather than holders of state.

Coordination-affecting state lives in substrate. Any state that affects coordination across executions is substrate-resident. Cell-internal counters become substrate-resident counts. Cell-internal caches that hold authoritative content become either substrate-recorded values (if authoritative) or derived views (if non-authoritative and regeneratable from substrate). Cell-internal session state becomes substrate-resident session records. The architectural pattern is that cells read substrate at invocation, substrate state shapes cell behavior, and cell outputs that affect coordination become substrate state.

Cell behavior is deterministic from substrate input. Two cells operating on identical substrate state produce consistent outputs (modulo allowed non-determinism from substrate-recorded sources such as time or external inputs). Hidden-state-driven non-determinism is not permitted.

The correction additionally requires distinguishing within-execution ephemeral state from persistent state — computed values, scratch calculations, and working memory used during a single execution are acceptable when cleared at execution end; persistence across executions is the failure mode — and maintaining an operational cell-state audit that re-invokes cells with identical substrate inputs and confirms consistent outputs, with inconsistencies treated as evidence of hidden state. Where read performance requires precomputed content, the architectural pattern places the derived view in an adjacent component, not in cell-internal state: substrate remains authoritative; the derived view is non-authoritative and regeneratable from substrate.

6. What the anti-pattern is NOT

Four legitimate patterns are commonly conflated with hidden state in cells and must be distinguished.

Not within-execution ephemeral computation. Cells using runtime variables, computed values, or working memory during a single execution are legitimate when this state is cleared at execution end. The anti-pattern arises specifically when state persists across executions. Within-execution computation is part of how cells execute behavior; it is not the failure.

Not cell runtime state for non-coordination concerns. Cells may have runtime state for operational concerns — debugging buffers, performance counters, logging state, tracing context — that does not affect coordination decisions. These are legitimate when they do not migrate coordination-relevant state into the cell. The anti-pattern is specifically the migration of coordination-affecting state into cells.

Not derived views supplying input to cells. A substrate-derived view supplying content to adjacent components, including cells, is legitimate when the view is non-authoritative and regeneratable from substrate. The anti-pattern is different: it is cell-internal caches or stores that hold coordination-relevant state without being derived views — that is, without being regeneratable from substrate and without acknowledging substrate as authoritative.

Not substrate-resident state read by cells at invocation. Cells reading substrate state at invocation use that state to inform behavior; this is the legitimate architectural pattern for how cells consume coordination state. The anti-pattern is different: it is holding coordination-relevant state inside the cell

across invocations rather than reading it from substrate at each invocation.

7. Operational test

A deployment exhibits hidden state in cells if any of the following are true at any time during the deployment's existence.

1. Cells maintain internal state — local variables, runtime caches, session state, instance data, configuration variables, internal buffers — that persists between executions and affects subsequent coordination decisions.
2. Cell behavior depends on cell-internal state not visible in substrate; identical substrate inputs may produce different cell outputs depending on cell-internal state.
3. Downstream operations consult cell outputs that reflect hidden state, in place of consulting substrate as the source of truth.
4. Cell-state changes occur without substrate-recording; coordination-relevant state transitions occur within cells without producing substrate writes.

Three sharpening properties operationalize the test for deployment review.

Cell-state-content-locus test. Verify operationally what state is held inside cells. Examine cell internal state across invocations; persistent coordination-relevant state inside the cell indicates the anti-pattern.

Cell-behavior-determinism test. Verify operationally whether cell behavior is deterministic from substrate state alone. Re-invoke cells with identical substrate state; inconsistent outputs across invocations indicate hidden state.

Hidden-state-inspection test. Verify operationally whether humans can identify what state shapes cell behavior. Attempt cell-state inspection through governance interfaces; inability to identify the determining state indicates the anti-pattern.

A deployment that fails any of (1)–(4) and any of the three sharpening properties exhibits the anti-pattern. The architectural correction in §5 specifies the operational changes required.

One-sentence test. If a deployment's cells maintain internal state — local variables persisting across invocations, runtime caches, session state, instance data, internal buffers — that operationally affects coordination decisions, and cell behavior depends on this hidden state in addition to substrate input, the deployment exhibits hidden state in cells; both the substrate-cell boundary and the substrate-as-source-of-truth commitments fail through the same mechanism.

8. Conclusion

Implementations under pressure to deliver stateful AI services consistently default to hidden state in cells because stateful services, closure-based processing, and session-based AI are common architectural patterns in surrounding engineering practice. The drift is steady because audiences understand "our cells maintain state for performance" as standard engineering without recognizing the architectural consequence: cells holding coordination-relevant state operationally determine coordination outside substrate authority. The downstream consequences manifest as cell behavior

varying with accumulated state, identical substrate state producing different cell outputs, retraceability gaps at hidden-state-driven moments, governance compromise, hidden-state-driven emergent behavior, cell-instance divergence in distributed deployments, operationally constrained recovery, and state-accumulation drift over time.

Naming hidden state in cells as a standalone anti-pattern — with the four operational components in §2, the violations in §3, the failure mode in §4, the architectural correction in §5, the four adjacent-pattern distinctions in §6, and the operational test with three sharpening properties in §7 — gives downstream readers a precise specification of the failure mode and its correction. The anti-pattern is the *general* cell-internal-state source-of-truth failure: it includes but is not limited to the agent-memory and LLM-context specific cases, and it compounds with the cell-as-substrate architectural-boundary failure when cells hold coordination-relevant state that is also operationally authoritative. Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should classify cell-internal-state configurations against the formalization here.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Anti-Pattern: Hidden State in Cells — A Standalone Formalization of the Cell-Internal-State Source-of-Truth Failure in the Coordination Knowledge Substrate Pattern*. May 6, 2026. ORCID: 0009-0004-8065-3235.